Relatório Técnico — Pipeline DevOps e Cloud com Kubernetes

Aluno: Daniel Moreira **Data:** 26 de abril de 2026 **Projeto:** ecommerce-microservices — user-service

---

1. Introdução

Este relatório documenta a implementação completa de um pipeline DevOps moderno para o microsserviço `user-service`, componente de um sistema de e-commerce baseado em arquitetura de microsserviços. O trabalho abrange desde a containerização com Docker até a orquestração com Kubernetes, passando por observabilidade com Prometheus/Grafana, CI/CD com Jenkins e testes de stress com k6.

O `user-service` é um serviço NestJS 10 com TypeScript que implementa autenticação JWT, registro de usuários e verificação de e-mail, utilizando PostgreSQL como banco de dados e Kafka para mensageria orientada a eventos.

---

2. Serviço Escolhido: user-service

2.1 Justificativa

O `user-service` foi selecionado por ser o microsserviço com a implementação mais completa do repositório (63 arquivos), com:

- Domínio rico (entidades, eventos, value objects)
- Casos de uso implementados (Register, Login, VerifyEmail)
- Infraestrutura completa (TypeORM, Kafka, JWT, Redis)
- Health controller funcional com verificação real de banco de dados

2.2 Endpoints HTTP

Método	Rota	Descrição
POST	`/api/v1/users/register`	Cadastro de novo usuário
POST	`/api/v1/users/login`	Login e emissão de JWT
POST	`/api/v1/users/verify-email`	Verificação de e-mail
GET	`/api/v1/users/:id`	Buscar perfil (JWT)
PUT	`/api/v1/users/:id/profile`	Atualizar perfil (JWT)
GET	`/health/live`	Liveness probe
GET	`/health/ready`	Readiness probe (verifica DB)
GET	`/metrics`	Métricas Prometheus

---

3. Containerização Docker

3.1 Dockerfile Multi-Stage

O Dockerfile foi construído com três estágios para minimizar o tamanho da imagem final e garantir segurança:

Stage	Nome	Propósito
1	`deps`	Instala dependências via `npm ci`
2	`builder`	Compila TypeScript e gera `dist/`
3	`runner`	Imagem de produção mínima, usuário não-root

Benefícios do multi-stage:

- Imagem final sem ferramentas de build (menor superfície de ataque)
- Usuário não-root (`appuser`) evita escalada de privilégios
- Cache de layers otimizado para rebuilds rápidos

3.2 .dockerignore

Exclui `node_modules` , `dist/` , arquivos de teste, `.env` e diretórios de infraestrutura para reduzir o contexto de build e evitar vazamento de dados sensíveis.

3.3 docker-compose.yml

O arquivo demonstra obrigatoriamente:

- **Bind mount**: `./services/user-service/src` montado em `/app/services/user-service/src` (modo somente leitura) — permite hot-reload em desenvolvimento
- **Named volume**: `postgres_data` persiste os dados do PostgreSQL entre `docker compose down/up`

Stack completa: PostgreSQL 16, Redis 7, Zookeeper, Kafka, Prometheus, Grafana.

3.4 Endpoint /metrics

O endpoint `/metrics` foi adicionado com a menor mudança possível: inclusão do pacote `prom-client` e criação de um `MetricsController` . Exporta:

- Métricas padrão do Node.js (CPU, memória, event loop lag, GC)
- `http_requests_total` (contador por método/path/status)
- `http_request_duration_ms` (histograma de latência)

---

4. Kubernetes

4.1 Estrutura de Manifests

```
k8s/
├── base/
│   ├── namespace.yaml          # Namespace ecommerce
│   ├── app-deployment.yaml     # Deployment (4 réplicas)
│   ├── app-service-nodeport.yaml # NodePort :30001
│   ├── deps.yaml               # PostgreSQL + Redis + Secrets + PVC
│   └── servicemonitor.yaml     # Prometheus ServiceMonitor
└── overlays/
    └── local/
        └── kustomization.yaml  # Kustomize para ambiente local
```

4.2 Deployment

- **4 réplicas** com `RollingUpdate` (maxSurge=1, maxUnavailable=0) para zero downtime
- **Liveness probe**: `GET /health/live` — reinicia container travado
- **Readiness probe**: `GET /health/ready` — verifica conexão com PostgreSQL antes de receber tráfego
- **Resources**: requests 256Mi/250m, limits 512Mi/500m
- **preStop hook**: aguarda 5s para drain gracioso de conexões

4.3 Dependências (deps.yaml)

Recurso	Tipo	Descrição
`user-service-secrets`	Secret	Senha do DB e JWT Secret (base64)
`postgres-pvc`	PVC	5Gi ReadWriteOnce para PostgreSQL
`postgres`	Deployment + Service ClusterIP	PostgreSQL 16 com healthcheck
`redis`	Deployment + Service ClusterIP	Redis 7 com healthcheck

4.4 Kustomize

O overlay `local` usa Kustomize para sobrescrever a tag da imagem e adicionar labels de ambiente sem duplicar YAML.

---

5. Observabilidade

5.1 kube-prometheus-stack

Instalado via Helm no namespace `monitoring` com:

- **Grafana**: NodePort 30300, PVC 2Gi
- **Prometheus**: ClusterIP, PVC 5Gi, retenção 7 dias
- **ServiceMonitorSelector**: coleta ServiceMonitors de todos os namespaces

5.2 ServiceMonitor

O `servicemonitor.yaml` configura o Prometheus Operator para fazer scrape do endpoint `/metrics` do user-service a cada 15 segundos.

5.3 Dashboard Grafana

Dashboard com 7 painéis:

1. **Pods Rodando** — contagem de pods em estado Running
2. **CPU Usage** — percentual de uso vs limite
3. **Memória Usage** — percentual de uso vs limite
4. **Requisições/s** — taxa por endpoint
5. **Latência P99** — histograma P99 por endpoint
6. **Taxa de Erros (5xx)** — gauge com thresholds visual
7. **Erros por Status Code** — série temporal por código HTTP

---

6. Jenkins CI/CD

6.1 Stages do Pipeline

```
Checkout → Docker Build → Docker Push → K8s Deploy → Smoke Test
```

Stage	O que faz
Checkout	Clona repositório, exibe últimos commits
Docker Build	Build multi-stage da imagem
Docker Push	Push com tags `:latest` e `:<build_number>`
K8s Deploy	`kubectl apply -k` + `rollout status` com timeout
Smoke Test	Verifica `/health/live` e `/health/ready` via NodePort

6.2 Credenciais

- `dockerhub-credentials` : username/password do Docker Hub
- `kubeconfig-minikube` : arquivo kubeconfig do minikube como Secret file

6.3 Subindo Jenkins

```
docker run -d --name jenkins --restart=unless-stopped \
  -p 8080:8080 -p 50000:50000 \
  -v jenkins_home:/var/jenkins_home \
  -v /var/run/docker.sock:/var/run/docker.sock \
  jenkins/jenkins:lts-jdk17
```

Senha inicial: `docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword`

---

7. Testes de Stress

7.1 k6

Script com ramp progressivo:

- 30s → 10 VUs (aquecimento)
- 1m → 50 VUs (carga moderada)
- 2m → 100 VUs (carga máxima)
- 30s → 0 VUs (desaquecimento)

Thresholds:

- P95 de latência < 2 segundos
- Taxa de erros < 5%

Fluxo testado por VU: health check → registro → login → métricas.

7.2 JMeter

Dois Thread Groups:

- **TG1**: 50 usuários, 20 iterações, ramp 30s — GET `/health/live`

- **TG2**: 30 usuários, 10 iterações, ramp 60s — POST `/api/v1/users/login`

Assertions: status 200, duração < 2s.

7.3 Quando Capturar Screenshot no Grafana

Durante o k6, quando os VUs atingirem 100 (aproximadamente 3min30s após o início):

1. Abra o painel "Requisições por Segundo"
2. Abra o painel "Latência P99"
3. Capture a tela mostrando os dois painéis ao mesmo tempo
4. Verifique que a taxa de erros está abaixo de 5%

---

8. Makefile

Targets disponíveis para automação:

Target	Descrição
`docker_build`	Build da imagem Docker
`docker_push`	Build + push para Docker Hub
`minikube_up`	Inicia minikube (4 CPUs, 6GB RAM)
`k8s_apply`	Aplica manifests via Kustomize
`k8s_status`	Status de todos os recursos
`obs_install`	Instala kube-prometheus-stack via Helm
`obs_status`	Status do namespace monitoring
`grafana_url`	Exibe URL e credenciais do Grafana
`stress_k6`	Executa teste de carga k6
`jenkins_up`	Sobe Jenkins via Docker
`jenkins_password`	Exibe senha inicial do Jenkins

---

9. Arquitetura Geral

```
┌──────────────────────────────────────────────────────┐
│                  Namespace: ecommerce                 │
│                                                       │
│  ┌──────────────────┐    ┌────────────┐              │
│  │   user-service   │    │ postgres   │              │
│  │   (4 réplicas)   │──▶│ ClusterIP  │              │
│  │   NodePort:30001 │    │  PVC 5Gi   │              │
│  └──────────────────┘    └────────────┘              │
│           │                                           │
│           │              ┌────────────┐              │
│           └──────────────▶│   redis    │              │
```

```
│                         │ ClusterIP │              │
│                         └───────────┘              │
└────────────────────────────────────────────────────┘
          │ /metrics scrape (15s)
          ▼
┌────────────────────────────────────────────────────┐
│                Namespace: monitoring               │
│                                                    │
│  Prometheus (ClusterIP, PVC 5Gi, retenção 7d)      │
│  Grafana (NodePort:30300, PVC 2Gi)                 │
└────────────────────────────────────────────────────┘
```

---

10. Conclusão

O pipeline implementado cobre o ciclo completo de DevOps:

1. **Containerização** com Docker multi-stage e segurança (usuário não-root)
2. **Orquestração** com Kubernetes (4 réplicas, probes, resources, PVC)
3. **Observabilidade** com Prometheus + Grafana (métricas, dashboards, alertas)
4. **CI/CD** com Jenkins (5 stages, credenciais seguras, smoke test automatizado)
5. **Testes de carga** com k6 e JMeter (thresholds objetivos, ramp progressivo)

Todas as práticas de 12-Factor App foram aplicadas: configuração via variáveis de ambiente, logs em stdout, processos stateless, graceful shutdown com SIGTERM.

GitHub: O projeto e todos os seus arquivos e evidências estão em:

https://github.com/danimoreira90/ecommerce-microservices